

DETAILED ANSWER SHEET

PAPER 1 – END SEM / FINAL PAPER | 23CSE306 – MACHINE LEARNING

Q1. Reinforcement Learning and Data Pre-processing

1. Reinforcement Learning (RL)

Reinforcement Learning is a machine learning paradigm in which an intelligent **agent** learns to make decisions by interacting with an **environment**. Instead of being given the correct answer for every input, the agent performs actions and receives feedback in the form of **rewards or penalties**. Its objective is to learn a policy that maximizes the expected cumulative reward over time.

- **Agent:** The learner or decision-maker.
- **Environment:** Everything with which the agent interacts.
- **State (s):** The current situation observed by the agent.
- **Action (a):** A decision or operation performed by the agent.
- **Reward (r):** Numerical feedback indicating how desirable an action was.
- **Policy (π):** A strategy mapping states to actions.
- **Value/Q-value:** Expected long-term usefulness of a state or state-action pair.

The learning loop is: **observe state** → **choose action** → **environment changes** → **receive reward** → **update policy**. Unlike supervised learning, RL must often balance **exploration** of new actions with **exploitation** of actions already known to perform well.

Autonomous car example

For an autonomous car, the **agent** is the driving controller. The **environment** includes roads, lanes, traffic signals, other vehicles, pedestrians, weather, road boundaries and the car's physical dynamics. A state may contain camera/LiDAR information, speed, position, heading and nearby obstacles. Actions may include steering, acceleration and braking.

The **reward function** guides safe driving. Positive rewards can be given for staying in the lane, maintaining safe speed, reaching the destination and smooth driving. Negative rewards or large penalties can be assigned for collisions, unsafe lane changes, traffic-rule violations, excessive braking or leaving the road. The agent gradually learns a policy that maximizes long-term safety and successful navigation rather than simply maximizing immediate speed.

2. Importance of data pre-processing and feature transformation

Real-world data is commonly incomplete, noisy, inconsistent and expressed in different units. Pre-processing converts raw data into a form suitable for learning. Without it, a model may learn misleading patterns or become overly influenced by irrelevant numerical differences.

- **Missing values:** They may need imputation or removal so that algorithms can operate reliably.
- **Noise and outliers:** Cleaning or robust transformation can prevent a few abnormal observations from dominating the model.
- **Scaling:** Standardization or normalization is essential for distance- and gradient-based algorithms such as KNN, K-Means and many neural networks.
- **Encoding:** Categorical variables must be converted into numerical representations, for example through one-hot encoding.
- **Feature transformation:** Log, square-root, polynomial or other transformations can reduce skewness, stabilize variance and capture non-linear relationships.

- **Feature quality:** Selection or extraction can reduce dimensionality, redundancy and overfitting.

Therefore, good pre-processing improves data quality, numerical stability, training efficiency, fairness of feature influence and ultimately the generalization performance of the model.

Q2 (OR). Unsupervised Customer Segmentation and Accuracy

1. Unsupervised learning approach for e-commerce personalization

Because the company has no explicit labels such as 'recommended product accepted' or 'customer rating', this is naturally an **unsupervised learning** problem. The company can represent every customer using behavioral features such as number of products viewed, categories visited, time spent, purchase frequency, average order value, recency of purchase, repeat purchases and browsing-to-purchase ratio.

A suitable pipeline is: **collect behavioral data** → **clean and scale features** → **optionally reduce dimensionality** → **cluster customers using K-Means, Hierarchical Clustering or DBSCAN** → **analyze and label the discovered segments**.

- **Frequent high-value buyers:** Can receive loyalty benefits and premium recommendations.
- **Frequent browsers but low purchasers:** May indicate price sensitivity or decision hesitation; targeted discounts may help.
- **Category specialists:** Customers repeatedly visiting one product category can receive category-specific recommendations.
- **Occasional or dormant customers:** Can be targeted with re-engagement campaigns.
- **New or exploratory users:** Can be shown broad recommendations until their preferences become clearer.

Unsupervised learning is appropriate because the groups are not predefined. The algorithm discovers natural behavioral structure in the data. These segments can then be used for personalization, marketing, inventory planning and customer-retention strategies.

2. Accuracy calculation

There are 100 images. The model correctly identifies all 80 cats, but incorrectly predicts all 20 dogs as cats.

```
Correct predictions = 80
Total predictions = 100
Accuracy = Correct predictions / Total predictions × 100
= 80 / 100 × 100 = 80%
```

Although the numerical accuracy is **80%**, the model is ineffective for balanced recognition because it completely fails to recognize the dog class. The dataset is imbalanced: simply predicting 'cat' for every image already gives 80% accuracy. Therefore, accuracy alone is misleading. Class-wise precision, recall, F1-score and a confusion matrix should also be examined. The dog recall is **0%**.

Q3. Least Squares Regression and Decision Tree vs KNN

A) Line of best fit: $Y = a + bX$

Given data: (1,50), (2,55), (3,65). For simple linear regression:

```
b = [nΣXY - (ΣX)(ΣY)] / [nΣX² - (ΣX)²]
a = ■ - bX■
n = 3
ΣX = 1+2+3 = 6
ΣY = 50+55+65 = 170
ΣXY = 50+110+195 = 355
ΣX² = 1+4+9 = 14
```

$$\begin{aligned}
 b &= [3(355) - (6)(170)] / [3(14) - 6^2] \\
 &= (1065 - 1020) / (42 - 36) \\
 &= 45/6 = 7.5 \\
 X_{\text{mean}} &= 6/3 = 2 \\
 \bar{y} &= 170/3 = 56.67 \\
 a &= 56.67 - 7.5(2) = 41.67
 \end{aligned}$$

Therefore, the estimated regression equation is:

$$Y = 41.67 + 7.5X$$

Interpretation: according to this fitted model, each additional hour studied increases the predicted score by approximately **7.5 marks**.

B) When to choose a Decision Tree over KNN

- Choose a **Decision Tree** when interpretability and explicit decision rules are important.
- It is useful when the relationship is non-linear and involves interactions between features.
- Decision Trees generally do not require feature scaling, while KNN is strongly affected by scale.
- For large prediction workloads, a trained tree can make predictions quickly; KNN may need to compare a new sample with many stored training examples.
- Trees can naturally handle mixed feature types after appropriate categorical processing and can provide feature-importance information.

KNN may instead be preferable when similar observations are expected to have similar outputs and the data has a meaningful distance measure. Its performance can degrade in high-dimensional data and it is sensitive to irrelevant features and scaling.

Q4 (OR). Ensemble Methods and Logistic Regression

1. Why use ensemble methods?

Ensemble learning combines predictions from multiple models to produce a stronger final predictor. The central idea is that different models may make different errors; combining them can reduce overall error.

- **Bagging:** Trains models on different bootstrap samples and aggregates predictions. Random Forest is a major example and mainly reduces variance.
- **Boosting:** Builds models sequentially, focusing more on previously difficult examples. Examples include AdaBoost and Gradient Boosting; these can reduce bias and create highly accurate predictors.
- **Voting:** Combines outputs through majority voting or probability averaging.
- **Stacking:** Uses a meta-model to learn how to combine predictions of several base models.

Advantages include improved generalization, robustness and accuracy. However, ensembles require more computation, may be harder to interpret and can still overfit if poorly designed or trained on weak/diversity-poor models.

2. Logistic Regression in real-life applications

Logistic Regression is primarily a **classification** algorithm. For binary classification it models the probability that an observation belongs to class 1 using the sigmoid function:

$$\begin{aligned}
 P(Y=1|X) &= 1 / [1 + e^{(-z)}] \\
 \text{where } z &= \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n
 \end{aligned}$$

Typical applications include spam detection, customer churn prediction, loan default/risk classification, fraud screening, disease-risk classification and marketing response prediction. It is useful when an interpretable probability and understandable relationship between features and outcome are desired.

Its strengths are simplicity, fast training, probabilistic output and interpretability of coefficients. Limitations include sensitivity to poor feature representation, difficulty capturing highly complex non-linear boundaries without engineered features, and possible performance limitations compared with advanced non-linear models.

Q5. Data Pre-processing and One Iteration of K-Means

1. Impact of preprocessing on clustering

Clustering is highly dependent on the representation of data because there are no target labels to correct poor feature influence. In distance-based methods such as K-Means, variables with larger numerical ranges can dominate the Euclidean distance.

- **Scaling:** Standardization ensures features such as income and age contribute comparably.
- **Outlier handling:** Important because K-Means centroids are means and can be strongly shifted by extreme values.
- **Missing-value treatment:** Prevents undefined or distorted distance calculations.
- **Feature selection:** Removes irrelevant dimensions that may hide the natural cluster structure.
- **Dimensionality reduction:** Can reduce noise and make clusters easier to visualize and identify.

Thus, preprocessing directly influences distances, cluster assignments, centroid locations and the interpretability of the final groups.

2. One K-Means iteration

Data: 42, 45, 47, 60, 62, 65, 80, 82, 85. Initial centroids: C1=45, C2=60, C3=80.

For one-dimensional data, Euclidean distance is simply the absolute difference from the centroid.

Value	Distance to C1=45	Distance to C2=60	Distance to C3=80	Assigned cluster
42	3	18	38	C1
45	0	15	35	C1
47	2	13	33	C1
60	15	0	20	C2
62	17	2	18	C2
65	20	5	15	C2
80	35	20	0	C3
82	37	22	2	C3
85	40	25	5	C3

Cluster assignments: C1 = {42,45,47}; C2 = {60,62,65}; C3 = {80,82,85}.

Updated C1 = $(42+45+47)/3 = 134/3 = 44.67$

Updated C2 = $(60+62+65)/3 = 187/3 = 62.33$

Updated C3 = $(80+82+85)/3 = 247/3 = 82.33$

Updated centroids after one iteration: C1 = 44.67, C2 = 62.33, C3 = 82.33.

Q6 (OR). K-Means with Two Clusters and a Real-World System

1. Apply K-Means

Values: {2,4,10,12,3,20,30,11,25}; initial centroids C1=4 and C2=11.

First assignment based on nearest centroid:

Value	$ x-4 $	$ x-11 $	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

After first assignment:

$C1 = \{2, 4, 3\}$; new centroid = $(2+4+3)/3 = 3$

$C2 = \{10, 12, 20, 30, 11, 25\}$; new centroid = $108/6 = 18$

After recomputing distances using centroids 3 and 18, the stable separation becomes:

Cluster 1 = {2,3,4,10,11,12}, centroid = 7

Cluster 2 = {20,25,30}, centroid = 25

A subsequent assignment remains unchanged, so these are the converged clusters.

2. Example: clustering-based customer segmentation system

A retail company can build a customer segmentation system using purchase frequency, recency, average order value, product categories and browsing activity.

- Collect customer interaction and transaction data.
- Clean missing records and remove extreme anomalies.
- Encode categorical information and standardize numerical features.
- Use K-Means; choose K using methods such as the elbow method or silhouette score.
- Analyze each cluster and assign business labels such as premium, loyal, price-sensitive, inactive or new customers.
- Use the segments for targeted offers, personalized recommendations and retention campaigns.

The system should periodically retrain because customer behavior changes over time.

Q7. Hard Voting, Soft Voting and Stacked Generalization

1. Hard voting

In hard voting, each classifier outputs a class label. The final class is selected by **majority vote**. For example, if three models predict A, A and B, the final prediction is A. Hard voting ignores the confidence associated with predictions.

2. Soft voting

In soft voting, classifiers output class probabilities. These probabilities are averaged or combined using weights, and the class with the highest combined probability is selected. Soft voting can be more informative because a highly confident model can influence the final decision, especially when weighted voting is used.

3. Stacking vs individual models

Individual models have their own strengths and weaknesses. A decision tree may capture non-linear rules, logistic regression may provide stable linear decisions and KNN may exploit local similarity. **Stacking** attempts to learn

how to combine these complementary behaviors.

Base models first generate predictions. A **meta-learner** is then trained using those predictions as input features. To avoid data leakage, the meta-training predictions are normally produced using cross-validation or out-of-fold predictions.

Stacking can improve accuracy when base models are diverse and make uncorrelated errors. However, improvement is not guaranteed. If base models make highly similar mistakes, or if the meta-model overfits, stacking may provide little benefit or even reduce generalization.

Q8 (OR). Random Forest vs Gradient Boosting and Stacking

1. Random Forest compared with Gradient Boosting Machines

Aspect	Random Forest	Gradient Boosting Machine
Training	Trees built largely independently using bagging	Trees built sequentially to correct earlier errors
Main effect	Primarily reduces variance	Can reduce bias and build a strong learner
Parallelism	Naturally parallelizable	Sequential nature is harder to parallelize
Robustness	Generally robust with less tuning	Often more sensitive to hyperparameters
Overfitting	Usually relatively resistant with enough diverse trees	Can overfit if too many trees or aggressive learning settings are used
Accuracy	Strong general baseline	Can achieve very high predictive accuracy with tuning
Speed	Often easier to train	May require more tuning and sequential computation

In practice, Random Forest is often chosen for a robust, relatively low-maintenance model, while Gradient Boosting is attractive when maximum predictive performance is important and careful tuning is feasible.

2. Stacked Generalization (Stacking)

Stacking is an ensemble technique in which multiple **base learners** are trained on the original data and their predictions are used as input to a **meta-learner**.

- **Level-0/Base models:** Examples may include Random Forest, SVM, Logistic Regression and KNN.
- **Meta-features:** Out-of-fold predictions generated by the base models.
- **Level-1/Meta-model:** Learns the best way to combine base predictions.

Training process: Split the training data into folds. For each fold, train each base model on the remaining folds and predict the held-out fold. Combine all held-out predictions to create an out-of-fold prediction matrix. Train the meta-model on this matrix and the true labels. Finally, retrain base models on the full training set and use their predictions as input to the trained meta-model for new samples.

The key benefit is that the meta-model can learn which models are more reliable in different situations. The major risks are computational complexity and overfitting if leakage-free validation is not used.

Q9. Recurrent Neural Networks and Activation Functions

1. RNN architecture and time-dependent data

A **Recurrent Neural Network (RNN)** is designed for sequential data. Unlike a standard feed-forward network, an RNN maintains a **hidden state** that carries information from earlier time steps.

$$h_t = f(Wx_t + W_h h_{t-1} + b)$$

$$y_t = g(W_y h_t + c)$$

Here, x_t is the input at time t , h_{t-1} is the previous hidden state, h_t is the updated memory and y_t is the output. The same weights are reused across time steps, allowing the model to process sequences of varying lengths.

Applications include language modeling, machine translation, speech recognition, sentiment analysis, time-series forecasting, anomaly detection and sequence generation. Basic RNNs can suffer from **vanishing and exploding**

gradients when learning long dependencies. LSTM and GRU architectures address this problem through gating mechanisms.

2. Activation functions

Function	Advantages	Disadvantages	Typical suitability
Sigmoid	Smooth; outputs 0–1; interpretable probabilities	Vanishing gradients; not zero-centered	Output layer for binary classification; gates
Tanh	Output centered around zero; strong self-gating	Still suffers from saturation and vanishing gradients	Hidden layers in older/RNN architectures
ReLU	Simple and computationally fast; reduces vanishing gradient issue	Outputs only positive values; can be inactive	Input and hidden layers in deep networks
Leaky ReLU	Allows small negative slope, reducing dead ReLU problem	Highly dependent on hyperparameter choice	Hidden layers where sparse activation is useful

Selection depends on network architecture and task. ReLU or its variants are commonly preferred in hidden layers, while sigmoid is particularly useful for binary probabilistic outputs. Multiclass classification commonly uses softmax at the output layer, although it was not one of the functions requested.

Q10 (OR). Recurrent Networks vs MLP and GD vs SGD

1. When to use recurrent networks instead of MLP

Use a **recurrent network** when the order of observations and dependencies across time or sequence positions are important. Examples include a sentence where word order changes meaning, sensor readings over time, stock/time-series sequences, speech and click streams.

An **MLP** is appropriate when inputs can be represented as a fixed-size feature vector and the relationship does not inherently require sequential memory. A plain MLP does not naturally preserve temporal order unless sequence information is explicitly engineered into features.

- **Use RNN/LSTM/GRU:** sequential or temporal data, variable-length sequences and context-dependent predictions.
- **Use MLP:** fixed tabular features, simpler feed-forward relationships and cases where sequence memory is unnecessary.

Therefore, the choice is determined by the structure of the data rather than by one model being universally superior.

2. Gradient Descent vs Stochastic Gradient Descent

Aspect	Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Data per update	Uses the entire training dataset	Uses one training example per update (or a small batch in common variants)
Update frequency	One update after computing full gradient	Very frequent updates
Computation	Expensive for very large datasets	Lower cost per update and scalable
Convergence path	Smooth and deterministic for fixed data/order	Noisy and fluctuating
Speed to useful solution	Can be slow on huge datasets	Often reaches a useful region quickly
Memory	May require processing all data for each update	Suitable for streaming or large-scale data
Local minima/saddles	Less random exploration	Noise can sometimes help escape poor regions

The general parameter update is:

$$\theta \leftarrow \theta - \eta \nabla J(\theta)$$

In batch Gradient Descent, ∇J is computed using all training examples. In SGD, the gradient is estimated from an individual example; therefore the update is faster but noisier. In modern machine learning, **mini-batch gradient descent** is widely used because it provides a practical balance between stable updates and computational efficiency.